Heart Failure Prediction API Deployment Report

Exploratory Analysis, Model Comparison, and Cloud Deployment with a Secure REST API

Arad Fadaei

Table of Contents

Chapter 1. Executive Summary

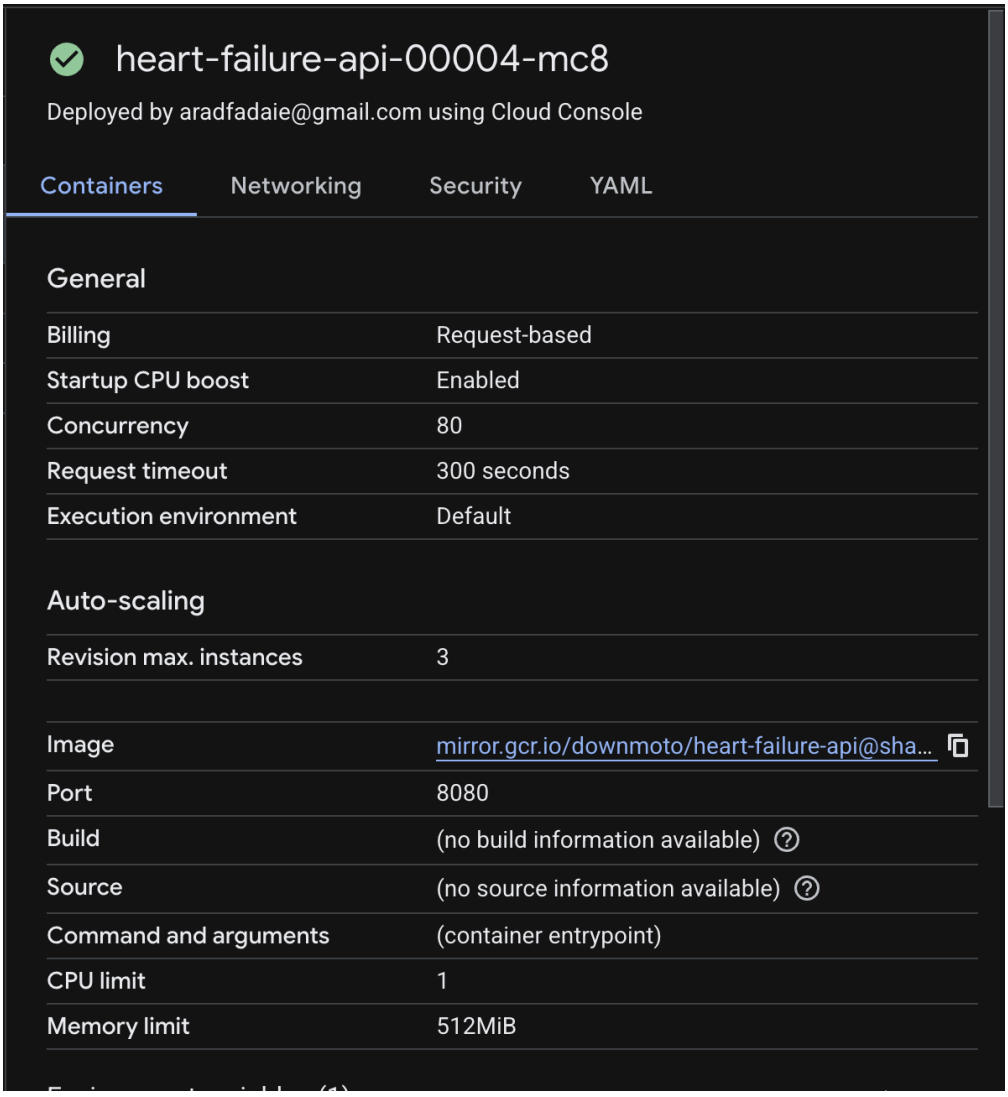
This project implements a complete machine learning deployment workflow for heart disease risk prediction using the Kaggle Heart Failure Prediction dataset. The final output is a containerized FastAPI service deployed on Google Cloud Run, with API key authentication and real-time inference.

The system follows a train-once, infer-many design: model training occurs offline, while the deployed service only loads saved artifacts and performs prediction. This satisfies the assignment constraint against training during inference requests.

Two models were trained and compared:

- Logistic Regression
- Random Forest Classifier

Random Forest was selected based on F1 score and overall balanced performance.



Chapter 2. Machine Learning Problem and Dataset

2.1. Machine Learning Problem

The task is binary classification: predict whether a patient is likely to have heart disease from structured clinical and demographic features.

Target variable:

- `HeartDisease` where `0` means no heart disease predicted and `1` means heart disease predicted.

2.2. Dataset Used

- Dataset source: Kaggle (`fedesoriano/heart-failure-prediction`)
- Local file used for training: `data/heart.csv`
- Data loading helper: `scripts/download_dataset.py`

2.3. Feature Set

The model uses these inputs:

- age
- sex
- chest pain type
- resting blood pressure
- cholesterol
- fasting blood sugar flag
- resting ECG
- max heart rate
- exercise angina
- oldpeak
- ST slope

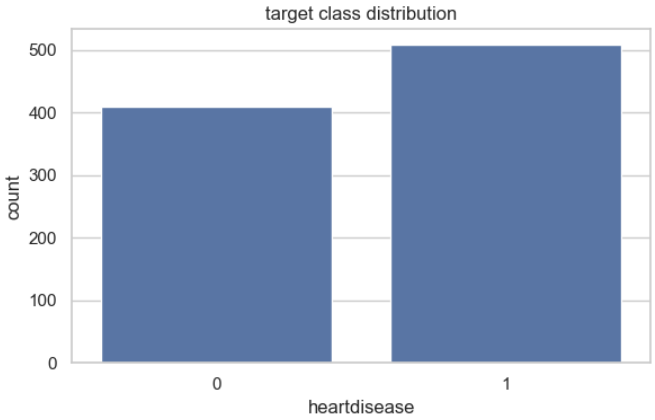
Chapter 3. Model Development Process

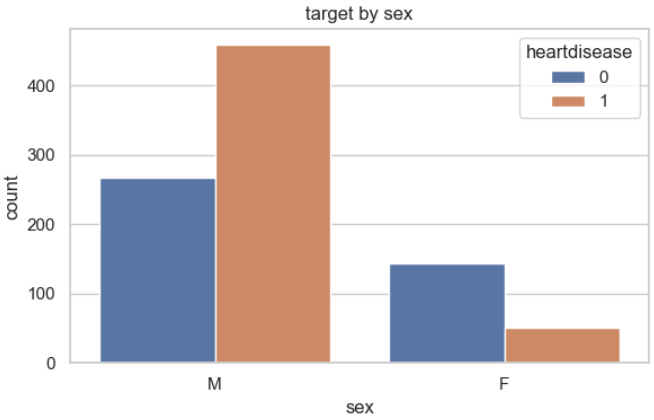
3.1. Exploratory Data Analysis (EDA)

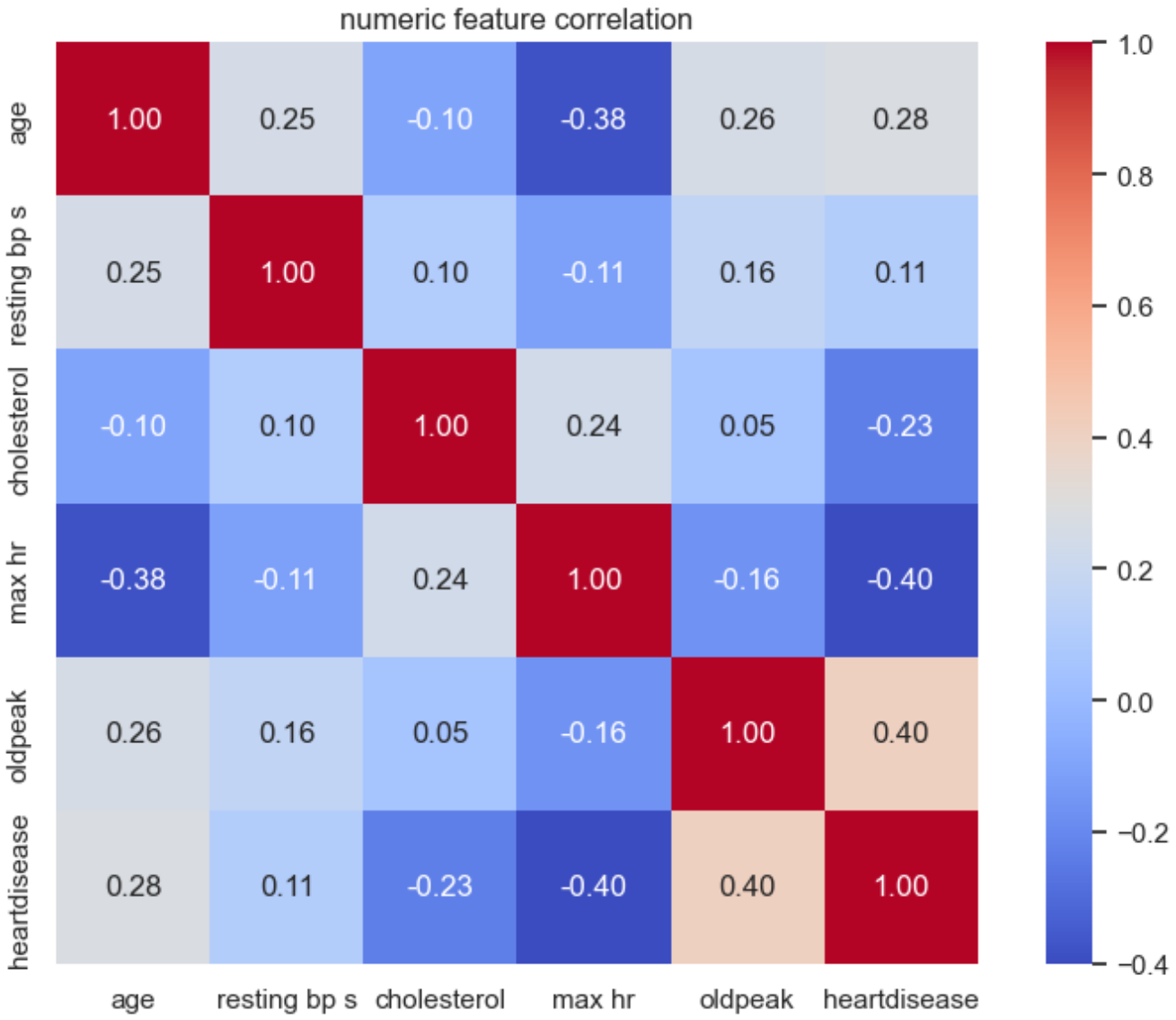
EDA was performed in `EDA.ipynb` to understand distribution, class balance, and relationships between variables.

EDA outputs include:

- target class distribution
- target split by sex
- numeric feature distributions by target
- boxplots by target
- correlation heatmap
- grouped risk rates by key categories







3.2. Preprocessing

Preprocessing is implemented with `ColumnTransformer` in `training/train.py`:

- numeric features: median imputation + standard scaling
- categorical features: mode imputation + one-hot encoding (`handle_unknown="ignore"`)

This ensures the same transformations are used during training and inference, reducing train/serve skew.

3.3. Training and Evaluation

Two models were trained on the same stratified split (`test_size=0.2`, `random_state=42`):

- Logistic Regression (`class_weight="balanced"`)
- Random Forest (`n_estimators=400`)

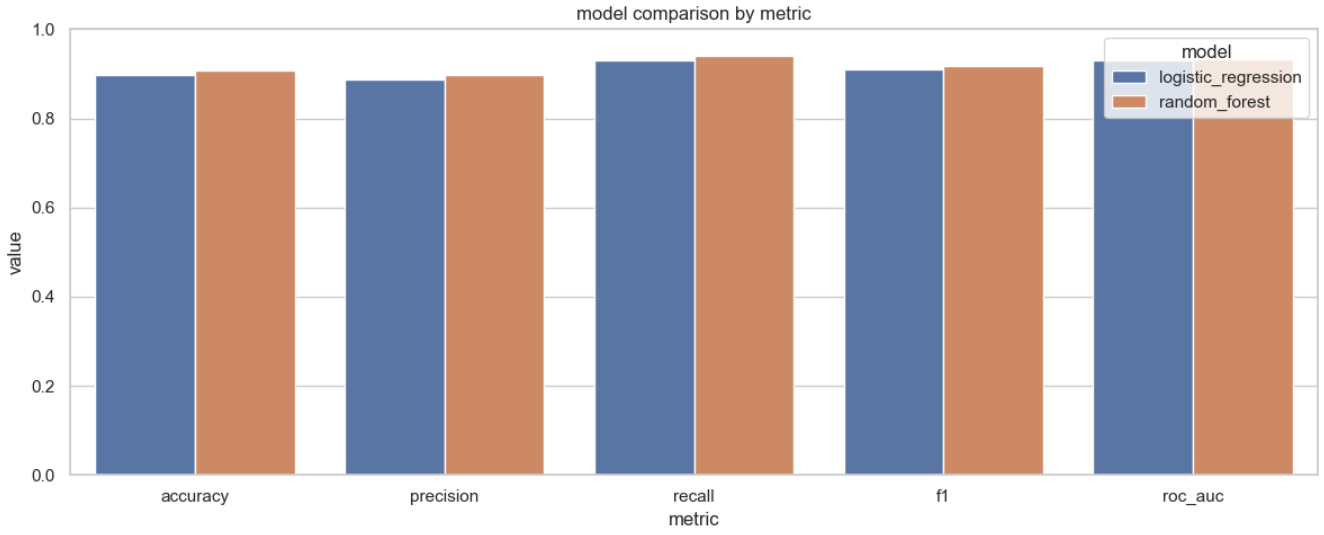
Evaluation metrics used:

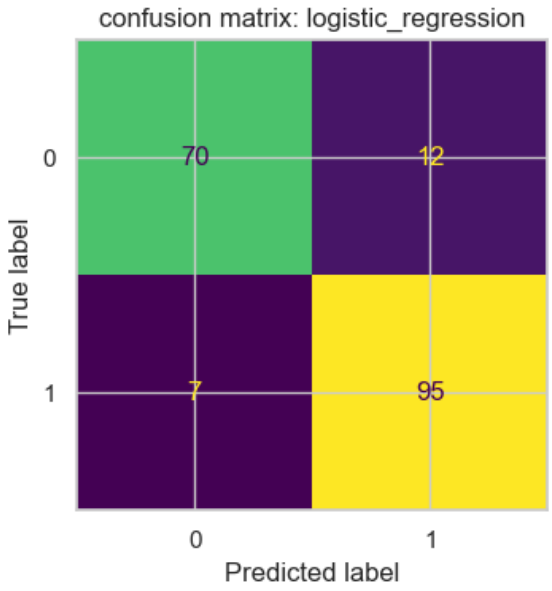
- Accuracy
- Precision
- Recall
- F1 Score
- ROC AUC

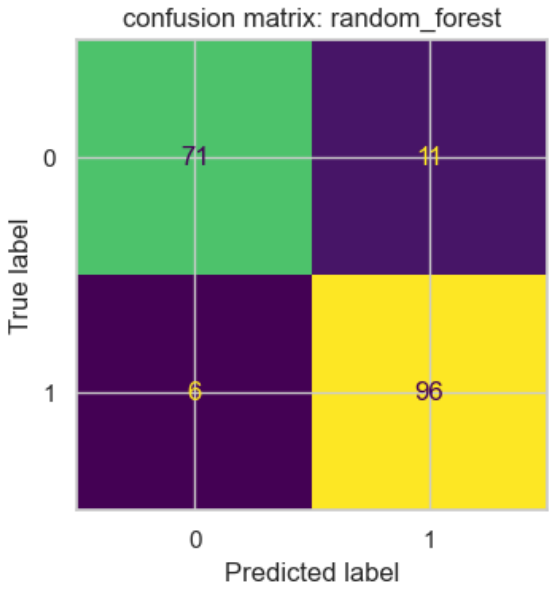
Evaluation summary:

Model	Accuracy	Precision	Recall	F1	ROC AUC
Logistic Regression	0.8967	0.8879	0.9314	0.9091	0.9296
Random Forest	0.9076	0.8972	0.9412	0.9187	0.9316

Selected model: Random Forest (highest F1).





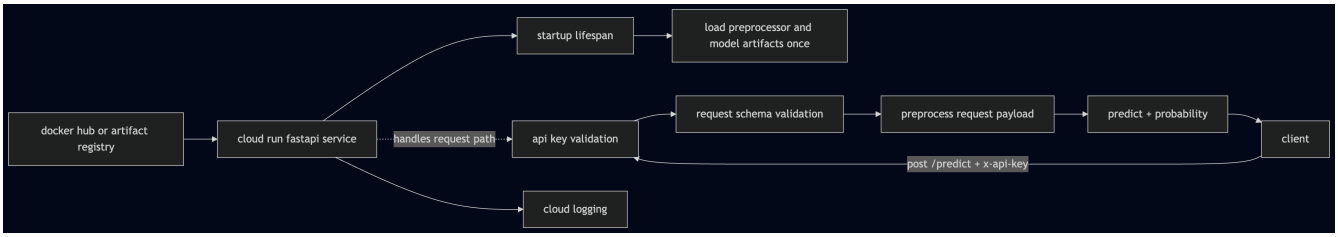


Chapter 4. Deployment Architecture Diagram and Cloud Services Used

4.1. Deployment Architecture Diagram

Architecture summary:

1. Client sends `POST /predict` with JSON payload and `x-api-key`
2. Cloud Run hosts the FastAPI container
3. API key is validated
4. Preprocessor and model artifacts are loaded and applied
5. Prediction and probability are returned
6. Logs are captured in Cloud Logging



4.2. Cloud Services Used

- Google Cloud Run: serverless container hosting for API
- Docker Hub image source: container image storage
- Cloud Logging: request and runtime logs

4.3. Containerization and API

Containerization is implemented with Docker:

- Base image: `python:3.12-slim`
- App server: `uvicorn src.main:app --host 0.0.0.0 --port 8080`
- Runtime secret: `API_KEY` environment variable
- Endpoint behavior:
  - `GET /health` for service checks
  - `POST /predict` for authenticated inference

Chapter 5. Challenges Faced and How They Were Overcome

5.1. Challenge 1: train/serve schema mismatch

Issue: source dataset columns and API request keys can differ.

Resolution: standardized column names in training and used explicit Pydantic aliases in API schemas.

5.2. Challenge 2: unseen categorical values at inference

Issue: new categorical values can break one-hot encoding.

Resolution: set `OneHotEncoder(handle_unknown="ignore")` in preprocessing pipeline.

5.3. Challenge 3: secure API key handling

Issue: keys should not be hardcoded in source or Dockerfile.

Resolution: read `API_KEY` from runtime environment variables and keep only `.env.example` as template.

5.4. Challenge 4: container architecture compatibility

Issue: Cloud Run deployment failed when image did not include `linux/amd64`.

Resolution: build and push amd64 image with `docker buildx --platform linux/amd64`.

Chapter 6. Conclusion and Future Improvements

This project successfully delivers a complete ML deployment pipeline, from EDA and model comparison to a live cloud-hosted API with authentication. The final implementation meets the assignment requirements and demonstrates an end-to-end operationalized workflow.

Potential future improvements:

- integrate Secret Manager for stronger secret governance
- add CI/CD for automated build, test, and deployment
- implement monitoring for drift and performance changes
- introduce model versioning and rollback strategy

Chapter 7. Screenshot Evidence of Working Deployment

1. Successful `POST /predict` response from deployed endpoint
2. Unauthorized `POST /predict` response (`401`) from deployed endpoint
3. Cloud logs showing request handling